



Designing UX for AI Interactions

Task

[Visit our website](#)

Introduction

As AI-powered features become more common in modern web applications, developers must think carefully about how users experience these systems. Integrating an AI model into an application is not only a technical challenge. It is also a user experience challenge.

Traditional web applications often behave in predictable ways. A user submits a form, clicks a button, or loads a page and receives an immediate response. AI systems behave differently. Responses may take longer to generate, outputs may vary between requests, and failures may occur due to network conditions, server limitations, or API restrictions.

Because of this uncertainty, users need more guidance during AI interactions than they would in many traditional interfaces. If an AI-powered feature provides no feedback while processing a request, users may assume the application has frozen. If an AI response fails silently, users may lose trust in the system entirely.

Good UX design reduces this uncertainty. A well-designed AI interface communicates what the system is doing, reassures users during delays, and helps them recover from failures gracefully. These design considerations are essential in applications such as AI chatbots, recommendation systems, AI writing assistants, intelligent search interfaces, and generative content tools.

In the previous lesson, you learned how to connect to AI APIs using JavaScript and process AI-generated responses. In this lesson, you will focus on the user experience layer that surrounds those interactions. You will learn how to design interfaces that feel responsive, transparent, and reliable when working with AI systems.

The Nature of AI Interactions

One of the defining characteristics of AI systems is that they are probabilistic rather than deterministic. This means that the same prompt may produce different outputs at different times. It also means that responses may occasionally contain inaccuracies, incomplete information, or unexpected behavior.

From a technical perspective, AI systems are often dependent on asynchronous operations. Requests are sent to remote servers where processing occurs before a response is returned. Depending on the complexity of the request and the load on the provider's infrastructure, this process may take several seconds.

These delays create a fundamentally different user experience from traditional front-end interactions.

Consider a standard calculator application. When a user clicks a button, the result appears instantly. The interaction feels predictable and mechanical. An AI-powered writing assistant, however, may need several seconds to generate a response. During that time, users need reassurance that the request is being processed correctly.

This is why UX design plays such an important role in AI applications. The interface must bridge the gap between unpredictable system behavior and user expectations.

Designing Effective Loading States

A loading state is a visual indication that the application is currently processing a request. In AI systems, loading states are especially important because AI-generated responses often take longer than standard application responses.

Without a loading state, users receive no indication that the system is working. This creates uncertainty and frequently results in repeated clicks, abandoned interactions, or frustration.

The purpose of a loading state is not only to indicate activity. It is also to improve the perceived responsiveness of the application. Even when processing time cannot be reduced, thoughtful loading feedback can make the interaction feel smoother and more intentional.

One of the simplest approaches is displaying a spinner while a request is processing.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>AI Loading State Example</title>
  <!-- CSS: Basic page styling and a reusable hidden class -->
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    .hidden {
      display: none;
    }
    #loading-spinner {
      margin-top: 15px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <!-- HTML: Button and Loading message shown to the user -->
  <h1>AI Response Generator</h1>
  <button id="generate-btn">
    Generate Response
  </button>
  <div id="loading-spinner" class="hidden">
    Generating AI response...
  </div>
  <!-- JavaScript: Show and hide the loading state -->
  <script>
    const button =
      document.getElementById("generate-btn");
    const spinner =
      document.getElementById("loading-spinner");
    button.addEventListener("click", () => {
      // Show the loading message when processing starts.
      spinner.classList.remove("hidden");
      // Simulate a delay, as if waiting for an AI API response.
      setTimeout(() => {
        spinner.classList.add("hidden");
      }, 3000);
    });
  </script>
</body>
</html>
```

```
});  
</script>  
</body>  
</html>
```

Although spinners are widely used, they provide limited information about what the system is doing. In AI interfaces, contextual feedback is often more useful because users benefit from understanding the nature of the processing task.

For example, an AI-powered recommendation system might display messages such as:

- “Analyzing your preferences...”
- “Generating recommendations...”
- “Searching for relevant results...”

These messages create a clearer mental model of the system’s behavior and reassure users that the delay is expected.

A simple implementation may look like this:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport"  
    content="width=device-width, initial-scale=1.0">  
<!-- CSS: Simple spacing for the interface -->  
  <style>  
    body {  
      font-family: Arial, sans-serif;  
      padding: 20px;  
    }  
    #status-message {  
      margin-top: 15px;  
    }  
  </style>  
</head>  
<body>  
  <!-- HTML: Status message area -->  
  <h1>AI Recommendation Tool</h1>  
  <button id="recommend-btn">  
    Generate Recommendations  
  </button>  
  <p id="status-message"></p>
```

```

<!-- JavaScript: Update the message shown to the user -->
<script>
  const recommendButton =
    document.getElementById("recommend-btn");
  const statusMessage =
    document.getElementById("status-message");

  recommendButton.addEventListener("click", () => {
    // Use specific wording so the user understands what is happening.
    statusMessage.textContent =
      "Generating AI recommendations...";
  });
</script>
</body>
</html>

```

This small addition can significantly improve usability because users understand that processing is actively occurring.

Preventing Duplicate Requests

AI APIs often involve usage costs and rate limits. If users repeatedly click a button while waiting for a response, the application may accidentally send multiple identical requests.

This creates several problems. It increases API costs, produces duplicate outputs, and may overload the interface with competing responses.

A common UX practice is disabling user controls while a request is processing.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
<!-- CSS: Basic page styling -->
<style>
  body {
    font-family: Arial, sans-serif;
    padding: 20px;
  }

  button:disabled {

```

```

        cursor: not-allowed;
        opacity: 0.7;
    }
</style>
</head>
<body>

<!-- HTML: Button that triggers the AI request -->
<h1>AI Content Generator</h1>

<button id="generate-btn">
    Generate
</button>
<p id="status-message"></p>
<!-- JavaScript: Disable the button while the request is running -->
<script>
    const button =
        document.getElementById("generate-btn");

    const statusMessage =
        document.getElementById("status-message");

    button.addEventListener("click", () => {

        // Prevent repeated clicks while processing is in progress.
        button.disabled = true;
        button.textContent = "Generating...";
        statusMessage.textContent =
            "Please wait while the AI processes your request.";

        // Simulate the time taken by an AI API call.
        setTimeout(() => {

            // Restore the button once processing is complete.
            button.disabled = false;
            button.textContent = "Generate";
            statusMessage.textContent =
                "Response generated successfully.";
        }, 3000);
    });
</script>
</body>
</html>

```

This approach improves both usability and system stability. It also communicates clearly that the application is already processing the request.

Building a Complete AI Loading Workflow

In practice, AI interactions usually combine multiple UX techniques together. A professional AI interface typically includes:

- loading indicators,
- status messaging,
- disabled controls,
- response handling,
- and recovery behavior.

The following example demonstrates a more complete AI interaction flow.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <!-- CSS: Styling for the input, status, and response areas -->
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      max-width: 700px;
    }
    textarea {
      width: 100%;
      min-height: 120px;
      margin-bottom: 12px;
    }
    button:disabled {
      cursor: not-allowed;
      opacity: 0.7;
    }
    #status-message {
      margin-top: 12px;
      font-weight: bold;
    }
    #response-container {
      margin-top: 20px;
    }
  </style>
</head>
<body>
```



```

padding: 15px;
border: 1px solid #ccc;
min-height: 80px;
}
</style>
</head>
<body>
  <!-- HTML: AI prompt input and response area -->
  <h1>AI Response Generator</h1>
  <textarea id="prompt-input"
    placeholder="Enter your prompt here"></textarea>
  <button id="generate-btn">
    Generate Response
  </button>
  <p id="status-message"></p>
  <div id="response-container"></div>
  <!-- JavaScript: Manage the complete AI request flow -->
  <script>
    const button =
      document.getElementById("generate-btn");
    const statusMessage =
      document.getElementById("status-message");
    const responseContainer =
      document.getElementById("response-container");

    button.addEventListener("click", async () => {
      const prompt =
        document.getElementById("prompt-input").value;
      // Set the loading state before sending the request.
      button.disabled = true;
      button.textContent = "Generating...";
      statusMessage.textContent =
        "AI is processing your request...";
      responseContainer.textContent = "";
      try {
        const response = await fetch(
          "https://api.example.com/ai",
          {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({ prompt })
          }
        );
      }
    });
  </script>

```

```
const data = await response.json();
responseContainer.textContent = data.reply;
statusMessage.textContent =
  "Response generated successfully.";

} catch (error) {
  statusMessage.textContent =
    "Something went wrong. Please try again.";
} finally {
  // Always restore the button after success or failure.
  button.disabled = false;
  button.textContent = "Generate Response";
}
});
</script>
</body>
</html>
```

Notice how the interface continuously communicates with the user throughout the interaction. The system never leaves the user wondering whether the application is functioning correctly.

Understanding Failure in AI Systems

All production systems experience failures, and AI applications are no exception. In fact, AI-powered systems often introduce additional points of failure because they rely heavily on external APIs, asynchronous communication, and large-scale infrastructure.

Common AI-related failures include:

Failure Type	Description
Network failures	Internet connectivity problems interrupt requests
Timeout errors	The AI provider takes too long to respond
Rate limiting	Too many requests are sent within a short period
Authentication failures	Invalid or expired API keys
Server overload	AI infrastructure experiences high demand
Invalid requests	Poorly formatted prompts or request bodies
Partial responses	The AI stops generating unexpectedly

Professional applications assume that failures will happen. Instead of attempting to eliminate every possible issue, developers design systems that fail gracefully.

Graceful failure means that users receive clear feedback, understand what happened, and are guided toward recovery.

Designing Effective Error Messages

Poor error handling damages user trust quickly. Technical error messages may make sense to developers, but they often confuse users.

For example, this message provides very little practical value:

```
500 Internal Server Error
```

A stronger message explains the issue clearly and suggests what the user should do next:

```
The AI service is temporarily unavailable. Please try again in a few moments.
```

Good error feedback should accomplish three goals:

1. Explain what happened.
2. Reassure the user that the issue is understood.
3. Provide a next step or recovery path.

This becomes especially important in AI applications because users may already feel uncertain about the reliability of the system.

Handling Errors with Try-Catch

JavaScript provides the `try...catch` structure for managing runtime errors during asynchronous operations.

This is particularly important when working with AI APIs because network requests can fail for many different reasons.

```
try {  
  const response = await fetch(apiUrl);  
  if (!response.ok) {  
    throw new Error("Request failed");  
  }  
  const data = await response.json();  
  console.log(data);  
} catch (error) {  
  console.error(error);  
}
```

The `try` block contains the code that may fail, while the `catch` block handles the error safely without crashing the application.

This allows developers to display helpful feedback and maintain a stable user experience even when problems occur.

Implementing Retry Functionality

Many AI-related failures are temporary. A request may fail because of a brief network interruption or a momentary spike in server traffic. In these situations, retry functionality improves resilience and reduces frustration.

One of the simplest approaches is allowing the user to manually retry the request.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <!-- CSS: Hide elements and style feedback -->
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }

    .hidden {
      display: none;
    }

    #status-message {
      margin-top: 15px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <!-- HTML: Main action button and retry button -->
  <h1>AI Retry Example</h1>
  <button id="generate-btn">
    Generate Response
  </button>
  <button id="retry-btn" class="hidden">
    Retry
  </button>
  <p id="status-message"></p>
  <!-- JavaScript: Show the retry button when a request fails -->
```

```

<script>
  const generateButton =
    document.getElementById("generate-btn");
  const retryButton =
    document.getElementById("retry-btn");

  const statusMessage =
    document.getElementById("status-message");
  async function generateResponse() {
    statusMessage.textContent =
      "Generating response...";
    retryButton.classList.add("hidden");
    try {
      const response = await fetch(
        "https://api.example.com/generate"
      );
      if (!response.ok) {
        throw new Error("Request failed");
      }
      statusMessage.textContent =
        "Response generated successfully.";
    } catch (error) {

      statusMessage.textContent =
        "The AI service is unavailable. Please try again.";
      retryButton.classList.remove("hidden");
    }
  }
  generateButton.addEventListener(
    "click",
    generateResponse
  );
  retryButton.addEventListener(
    "click",
    generateResponse
  );
</script>
</body>
</html>

```

This creates a smoother recovery experience because users can retry immediately without refreshing the page or re-entering information.

Automatic Retry Systems

Some applications retry failed requests automatically. This approach is common when failures are likely to be temporary and short-lived.

The following example retries a request up to three times before reporting failure. It is shown as a full page so that you can see how automatic retry logic connects to the interface.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <!-- CSS: Basic layout and feedback styling -->
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    #status-message {
      margin-top: 15px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <!-- HTML: Button and status message -->
  <h1>Automatic Retry Example</h1>
  <button id="generate-btn">
    Generate Response
  </button>
  <p id="status-message"></p>
  <!-- JavaScript: Retry failed requests a limited number of times -->
  <script>
    const generateButton =
      document.getElementById("generate-btn");

    const statusMessage =
      document.getElementById("status-message");

    async function fetchWithRetry(
      url,
      options,
```

```

    retries = 3
  ) {

    for (
      let attempt = 1;
      attempt <= retries;
      attempt++
    ) {

      try {

        statusMessage.textContent =
          `Attempt ${attempt} of ${retries}...`;

        const response =
          await fetch(url, options);

        if (!response.ok) {
          throw new Error("Request failed");
        }

        return await response.json();

      } catch (error) {

        console.log(
          `Attempt ${attempt} failed`
        );

        if (attempt === retries) {
          throw error;
        }
      }
    }
  }

  generateButton.addEventListener("click", async () => {

    try {

      await fetchWithRetry(
        "https://api.example.com/generate",
        {
          method: "POST"
        }
      )
    }
  })
}

```



```
    );  
  
    statusMessage.textContent =  
        "Response generated successfully.;"  
  
    } catch (error) {  
  
        statusMessage.textContent =  
            "The request failed after several attempts.;"  
    }  
  
    });  
</script>  
  
</body>  
</html>
```

Although retry systems improve reliability, they must be implemented carefully. Infinite retry loops can increase API costs, overload servers, and create poor user experiences.

For this reason, retry systems typically include retry limits and delays between attempts.

Managing User Expectations

One of the most important responsibilities in AI UX design is expectation management.

Users often assume AI systems are more intelligent or reliable than they actually are. Poor UX design can reinforce unrealistic expectations, leading to disappointment when the system produces incorrect or inconsistent results.

Good interfaces acknowledge the strengths and limitations of AI systems openly. Rather than presenting AI as perfectly accurate, professional systems communicate that outputs are generated dynamically and may require human review.

For example, many AI writing tools include notices explaining that generated content should be verified before publication.

This approach improves credibility because the system appears transparent rather than misleading.

Conversational UX Patterns

Many AI applications use conversational interfaces such as chatbots and AI assistants. These interfaces require additional UX considerations because users expect interactions to feel responsive and natural.

One important pattern is the typing indicator. This creates the impression that the AI is actively generating a response rather than leaving the interface inactive.

```
AI is typing...
```

Another increasingly common pattern is streaming responses, where generated text appears gradually instead of all at once.

Streaming improves perceived performance because users begin receiving information immediately rather than waiting for the complete response. It also creates a more conversational experience that feels interactive and dynamic.

These patterns are widely used in systems such as:

- AI chat assistants,
- customer support bots,
- AI coding assistants,
- and recommendation interfaces.

Accessibility in AI Interfaces

AI interfaces should remain accessible to all users, including users who rely on assistive technologies. Dynamic AI interactions can create accessibility challenges because content changes frequently and asynchronously. Without proper accessibility support, screen readers may not announce status updates or newly generated content correctly.

One common solution is using ARIA live regions.

```
<div aria-live="polite"  
      id="status-message">  
</div>
```

The `aria-live` attribute informs assistive technologies that the content may update dynamically and should be announced to the user.

Accessible AI interfaces should also include:

- clear button labels,
- keyboard navigation support,
- sufficient color contrast,
- and feedback that does not rely solely on color.

Accessibility is not an optional enhancement. It is a core component of professional interface design.

Real-World AI UX Design

Modern AI products provide excellent examples of these principles in practice. Chat-based AI assistants often use streaming responses, conversational layouts, typing indicators, and retry functionality to create smoother interactions.

AI coding assistants provide inline suggestions gradually so that the interface remains non-blocking and responsive. AI image generation tools typically include progress indicators, queue status updates, and generation notifications because image creation may take significant time.

Although these applications differ in purpose, they share a common design philosophy: continuously communicate system status and reduce user uncertainty.



Extra resource

Before completing the practical task, open the `ai-prompt-generator-example` folder in your task folder. Review the `index.html`, `style.css`, and `script.js` files to see how loading states, retry functionality, and user feedback work together in a simple AI-powered interface.

This example demonstrates how multiple UX concepts combine to create a smoother and more resilient AI interaction.

Instructions



Take note

The task(s) below is/are **auto-graded**. An auto-graded task still counts towards your progression and graduation. Give it your best attempt and submit it when you are ready.

When you select “Request Review”, the task is automatically complete, you do not need to wait for it to be reviewed by a mentor.

You will then receive an email with a link to a model answer, as well as an overview of the approach taken to reach this answer.

Take some time to review and compare your work against the model answer. This exercise will help solidify your understanding and provide an opportunity for reflection on how to apply these concepts in future projects.

Once you’ve done that, feel free to progress to the next task.



Auto-graded task

You are building a simple AI study assistant for a learning platform. A student should be able to enter a topic and request a short study tip. The interface does not need to connect to a real AI API. Instead, you may simulate the AI response using `setTimeout()`.

Your final application should feel like a real AI-powered interaction, even though the response is simulated.

1. Create a small web application using three files:

```
index.html  
style.css  
script.js
```

2. Your application must allow the user to enter a prompt or topic, click a button to generate a response, and view the result on the page.
3. The interface must include a clear loading state while the response is being generated. During this time, the submit button must be disabled so the user cannot send duplicate requests. The button text should also change to show that the system is processing the request.
4. Your application must also include error handling. Simulate at least one possible failure state. For example, you may randomly decide whether the request succeeds or fails. When a failure occurs, the user must see a clear error message and a retry button.
5. The retry button should allow the user to run the request again without refreshing the page.
6. Your application should also include meaningful feedback for success, loading, and error states. These messages should be written in plain language that a non-technical user can understand.

Minimum requirements

Your submission must include:

1. A prompt input field or text area.
2. A generate button.
3. A visible loading state.
4. Disabled button behavior while the request is processing.
5. A generated response area.
6. A clear error message when the simulated request fails.
7. A retry button that appears only after failure.
8. Accessible feedback using `aria-live`.
9. Separate `index.html`, `style.css`, and `script.js` files.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback [form](#) to help us ensure we provide you with the best possible learning experience.
